

Who Dropped the Packet?



Solving K8s Network Mysteries in Real-Time

Neeraj Bhatt

Senior Software Engineer, OVN
Engineering

Red Hat India

DevConf.CZ 2026

Ramesh Sahoo

Principal Software Engineer, OVN
Engineering

Red Hat India

DevConf.CZ 2026

About Me

Neeraj Bhatt

Senior Software Engineer, OVN Core
Development at Red Hat

- 10+ years of OpenShift networking experience
- Previously in OCP networking support — thousands of sosreports analyzed
- Now contributing to OVN-Kubernetes upstream development



"I've seen every way a packet can disappear."

Kubernetes Networking is Complex

A production cluster has many layers:

Pod network: veth pairs, OVN switches

Service network: ClusterIP, Load Balancing

Overlay network: Geneve tunnels

Policies: OVN ACLs, conntrack zones

DNS: CoreDNS, service discovery

A single HTTP request touches:

Pod veth → br-int → OVN switch → OVN router → DNAT

→ Geneve encap → dest node → de-encap → dest pod

+ conntrack & NetworkPolicy at **every hop**

A packet can drop silently at ANY of these layers.

The error is always just: `'connection timed out.'`

What You See When It Breaks

Application symptoms (misleading):

'connection timed out'

is it DNS? routing? policy? conntrack?

'connection refused'

is the pod down? or is DNS broken?

'intermittent 502 errors'

depends on node, time, or payload size?

What you check first:

`oc get pods` — Running, Ready 1/1 ✓

`oc get endpoints` — exists ✓

`oc get svc` — service exists ✓

`oc logs` — app says 'timeout' (not helpful)

The gap: everything LOOKS fine, but packets vanish.

Traditional Tools — What They CAN and CAN'T Do

```
tcpdump -i any port 5000
```

Sees packets on ONE interface. Cannot see kernel-internal drops.

```
conntrack -S / conntrack -L
```

Shows counters and entries. No per-pod, per-flow attribution.

```
ovn-trace / ovs-appctl ...
```

Simulates the pipeline. Cannot simulate conntrack STATE.

```
ovs-ofctl dump-flows br-int
```

5000+ rules. Which one is relevant? Manual analysis.

The Blind Spot:

None of these tools can tell you: 'this packet was dropped by conntrack at this time, from this pod.'

Enter eBPF — Visibility INSIDE the Kernel

What is eBPF?

Extended Berkeley Packet Filter

- Safe programs running INSIDE the Linux kernel
- No kernel module, no reboot, no app changes

Safety First: The eBPF verifier ensures programs can't crash the kernel.

Unique Capabilities

- **Attach to TC hook:** Visibility across every interface and node.
- **kfree_skb tracepoint:** Fires exactly when the kernel DROPS a packet.
- **Exact Drop Reason:** Reports CT_TABLE_FULL, CT_INVALID, etc.
- **Metadata Extraction:** Captures ~100 bytes/packet without full payload.
- **Always-on:** Minimal overhead (~0.08 CPU total across 6 nodes at 3K flows/s; scales with cluster size).

Network Observability Operator Architecture

Data Pipeline (Top to Bottom)

OCP Console

Observe -> Network Traffic

Loki

Stores enriched flow logs

flowlogs-pipeline

Collects and enriches IPs to pod names + namespaces

eBPF Agent

DaemonSet — one pod per node

Linux Kernel

TC hook + kfree_skb tracepoint



Zero Code Required

You configure **ONE YAML**
(FlowCollector CR).

You do **NOT** write eBPF code.

From Raw Kernel Data to Actionable Insights

What the eBPF agent captures (raw):

```
src=10.128.2.15 dst=10.128.3.22 port=5000  
proto=TCP bytes=1247  
drop_reason=6 dns_rcode=0 rtt_ns=1500000
```



What FLP enriches it to (actionable):

Source: frontend (namespace: demo-app, node: worker-1)

Dest: backend-api (namespace: demo-app, node: worker-2)

Port: 5000 (http-alt)

Drop: CT_TABLE_FULL (conntrack table exhaustion)

DNS: NoError, 2ms latency

RTT: 1.5ms

The enrichment is what makes NOO practical.

Without it: you see IP addresses and numbers. **With it:** pod names, plain English drop reasons, and DNS query names.

Network Traffic View

1. Traffic Flow Table

- Filter by namespace, pod, node, protocol
- Sort by bytes, packets, drops — find the noisy neighbor

2. Drops View

- See exactly WHERE and WHY packets are dropped
- Drop reason: CT_TABLE_FULL, CT_INVALID, NetworkPolicy, etc.

3. DNS Tracking View

- Every DNS query: name, response code, latency
- Catch SERVFAIL, NXDOMAIN, timeouts instantly

4. Topology View

- Visual graph of pod-to-pod traffic
- See communication patterns at a glance

The FlowCollector CR: Your Only Configuration

The ONLY YAML you write:

```
apiVersion: flows.netobserv.io/v1beta2
kind: FlowCollector
spec:
  agent:
    type: eBPF
    ebpf:
      sampling: 1
      privileged: true
      features:
        - PacketDrop
        - DNSTracking
        - FlowRTT
        - NetworkEvents
```

Key Capabilities Enabled



PacketDrop

kfree_skb — identifies specific drop reasons



DNSTracking

Port 53 — correlates queries with responses



FlowRTT

TCP handshake — measures network latency



NetworkEvents

Correlates flows with OVN NetworkPolicies

✦✦ **After** `oc apply`: operator deploys DaemonSet & loads eBPF into kernel automatically.



Case Study: 3 Weeks → 15 Minutes

The Problem

Intermittent Timeouts

- 200+ microservices, timeouts during peak hours (5-30s failures)
- Support team spent 3 WEEKS with tcpdump, OVN traces, and sosreports

Traditional Tools Failed

tcpdump:

SYN no SYN-ACK — incorrectly suggested 'pod must be down'

conntrack -S:

insert_failed=847 — viewed as 'just a counter' without context

OVN trace:

'output to port' — pipeline falsely indicated ALLOW

NOO Resolution

Solved in 15 Minutes

- **Identified 5 service pairs dropping on worker-3 due to CT_TABLE_FULL**
- **Fix:** Increased `nf_conntrack_max` on affected nodes

Live Demo: Network Observability

Environment: OCP 4.21 | NOO 1.11 | eBPF Enabled

Target Application

Bookstore App

Frontend → Backend-API → Database

Demo Scenario

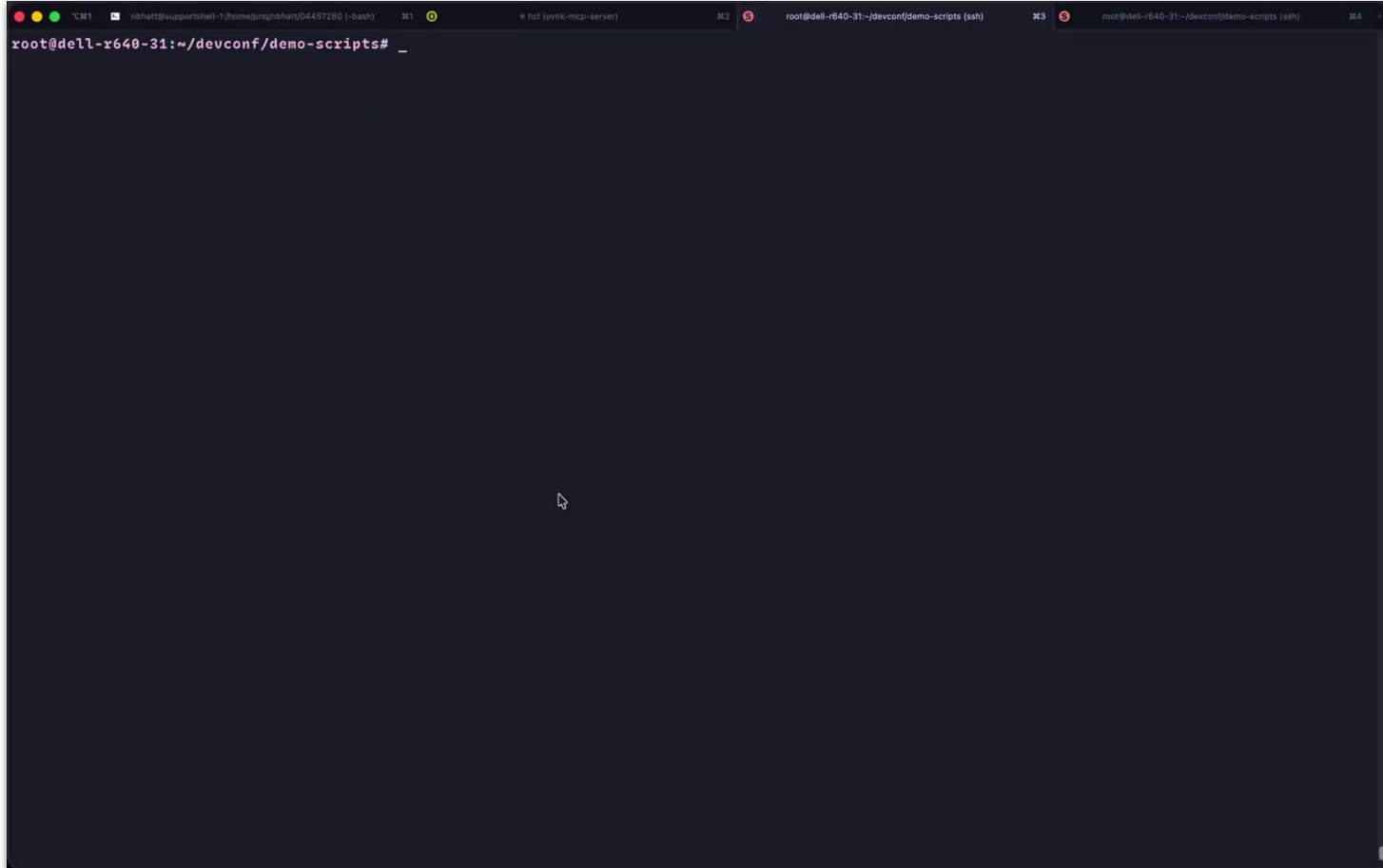
Debugging Connectivity & Policy Drops

The Issue

System State: Health: 200 OK | Database: 8 books, healthy

Failure: **Books Fetch FAILS**

Diagnosis: OCP Console shows drops on **backend-api** → **database** NetworkPolicy (Port 5432)



Understanding Sampling in NOO

What is sampling?

The eBPF agent sees **EVERY** packet. Sampling controls how many it **EXPORTS**.

sampling: 1 100% (every packet) —Best for demos and active troubleshooting in production

sampling: 50 2% (1 in 50) —Standard for production

sampling: 100 1% (1 in 100) —Required for high-traffic

Why not always use sampling: 1?

- At 10Gbps, nodes process **~1M packets/sec**
- Exporting all = **~100MB/s** of flow data per node
- Without Kafka, pipeline may struggle; use Kafka & sized Loki for production

Why sampling: 50 is still useful:

- **Patterns:** 100 drops → 2 drops visible
- **CT_TABLE_FULL:** 1000 pkts → ~20 visible
- **DNS Failures:** DNS: subject to sampling — use flow filter on port 53 for full capture
- **Reason Codes:** Even 1 sampled drop reveals the cause

NOO Architecture — Deep Dive



1. eBPF Agent

Loads programs into kernel at boot (DaemonSet)

TC hook: attached to EVERY interface

kfree_skb: kernel drop tracepoint tracking

Export flow records to FLP (default: 5s)



2. Pipeline (FLP)

Receives raw flow records from eBPF agents

Enrichment: queries K8s API for metadata

Aggregation: merges duplicate flows

Exports to Loki and Prometheus



3. Storage

Loki

Flow logs with labels for fast querying

Prometheus

Metrics for alerting and graphing



4. Console Plugin

Queries Loki and Prometheus to render intuitive, real-time observability dashboards for network flows.

eBPF Features — What Each One Does

PacketDrop

kfree_skb tracepoint

Fires when kernel DROPS a packet.
Reports reason code.

**CT_TABLE_FULL, CT_INVALID,
NETFILTER_DROP, PKT_TOO_BIG**

*Requires: privileged: true
(CAP_SYS_ADMIN)*

DNSTracking

TC hook, port 53

Parses DNS query names from
UDP/TCP port 53 packets.

- Captures: response code & latency
- Max 32 bytes of domain name per query

FlowRTT

TC hook, TCP handshake

Measures SYN → SYN-ACK timing to
determine connection RTT.

*Reports as `TimeFlowRttNs` in flow
records.*

eBPF Features — NAT Tracking, IPsec, TLS

PacketTranslation

Captures pre-NAT and post-NAT source/destination IPs + ports

Before DNAT

dst=172.30.100.50 (ClusterIP)

After DNAT

dst=10.128.2.22 (Pod IP)

Useful for debugging Service load balancing decisions

Security & Tracking

IPSec:

Tracks flows between nodes encrypted with IPsec. Shows which flows are encrypted vs. plaintext.

TLSTracking:

Detects TLS handshakes in flow data. Shows which flows use TLS and which don't.

UDNMapping [GA](#)

Maps flows to User Defined Networks (secondary interfaces).

Preview Features

EbpfManager Dev Preview

Delegates loading of eBPF programs to bpfman operator.

Custom Programs Not Supported

bpfttrace, bcc, Cilium Hubble.

NetworkEvents [OVN Integration](#)

Correlates flows with OVN-K NetworkPolicy evaluations.

Shows **MATCHED/DROPPED** rules.

Requires: OVN-K Observability & privileged

How eBPF + NOO Found It

The Investigative Chain

- 1. Entry:** Packet enters kernel → TC hook captures metadata
- 2. Pipeline:** Traverses OVN pipeline → forwarded (correct)
- 3. The Drop:** Hits conntrack → CT_INVALID → kernel drops it
- 4. Capture:** kfree_skb tracepoint FIRES → captures drop reason
- 5. Export:** eBPF agent reads ring buffer → exports flow record
- 6. Enrich:** FLP maps IP to 'backend-api on worker-3'
- 7. Visual:** Loki stores → Console displays the insight

Instant Visibility

Open the dashboard, filter by drops, and the answer is on your screen in **5 seconds**.

Why it matters:

"The eBPF kfree_skb tracepoint is the ONLY place in the kernel that reports WHY a packet was dropped."

Impact: Real Customer Results

Traditional Approach

2-20 DAYS

Average time to find root cause

Complex Tooling

tcpdump, sosreports, must-gathers, manual OVN analysis

High Resource Cost

3+ engineers, multiple escalations often required

eBPF Approach (NOO)

5-30 MINUTES

Average time to find root cause

Unified Interface

One browser tab: OCP Console → Network Traffic

Efficiency

1 engineer, no escalation needed

Common Issues Caught: Conntrack exhaustion, NetworkPolicy misconfig, DNS failures, MTU mismatches



Key Takeaways

1. Multi-Layer Complexity

Kubernetes networking involves 10+ hops through OVN, conntrack, tunnels, and NAT.

2. Tooling Limitations

Traditional tools like tcpdump see individual hops but miss the full picture.

3. eBPF Kernel Insight

eBPF sees inside the kernel via kfree_skb tracepoints to find drop reason codes.

4. Practical eBPF with NOO

NOO provides dashboards and storage from raw kernel data with a simple YAML config.

Resources & References



Official Documentation

docs.redhat.com — OCP 4.22 Network Observability



Upstream Project

github.com/netobserv (operator, agent, pipeline)
FlowCollector API: [.../FlowCollector.md](#)



Learn & Try eBPF

ebpf.io — Community Resource
demo.redhat.com — Live Sandbox



This Talk (Repo)

github.com/newera-begins/devconf_2026
Includes scripts, README, and sample outputs

Questions?

Neeraj Bhatt

Senior Software Engineer, OVN Core Development, Red Hat India

Let's solve your network mysteries together.



PacketTranslation

Captures pre-NAT and post-NAT source/destination IPs + ports

Before DNAT:

dst=172.30.100.50 (ClusterIP)

After DNAT:

dst=10.128.2.22 (Pod IP)

Useful for debugging Service load balancing decisions

Security & Tracking

IPSec:

Tracks flows between nodes encrypted with IPsec. Shows which flows are encrypted vs. plaintext.

TLSTracking:

Detects TLS handshakes in flow data. Shows which flows use TLS and which don't.

UDNMapping (GA)

Maps flows to User Defined Networks (secondary interfaces).

Preview Features

EbpfManager (Developer Preview)

Delegates loading of NOO's OWN eBPF programs to bpfman operator. Central manager for cluster-wide eBPF tools.

Custom Programs Not Supported

Use bpftrace, bcc, or Cilium Hubble for custom scripts.

Extension: Build custom GRPC collector for NOO flow data.

NetworkEvents (OVN integration)

Correlates flows with OVN-K NetworkPolicy evaluations.

Shows WHICH policy rule matched/dropped a flow.

Requires: OVN-K Observability feature

Requires: privileged: true.